

Milk Collection Management Platform

UI/UX System Requirements & AI Build Specification

Production-oriented design specification for a beautiful, highly interactive, responsive and user-friendly multi-tenant milk collection platform.

Document	UI/UX System Requirements
Platform	Multi-tenant Milk Collection Management
Frontend	Next.js + TypeScript + Tailwind CSS
Mobile focus	Android collector workflow
Backend context	NestJS + PostgreSQL + Prisma + Keycloak
Status	Design and implementation baseline

Core principle: make the application feel simple even though the underlying system is sophisticated.

1. Product Design Objectives

- Extremely simple workflows
- Very fast data entry
- Excellent mobile experience
- Excellent Android collector experience
- Clear visual feedback
- Strong information hierarchy
- Minimal cognitive load
- Professional enterprise appearance
- Excellent accessibility
- Beautiful dashboards without unnecessary visual clutter
- Strong offline-state visibility
- Clear tenant and cooler context
- Safe confirmation for destructive or financially sensitive actions
- Consistent interaction patterns throughout the system

Design rule: Do not sacrifice usability for visual effects. The product should feel like a modern SaaS platform, not a generic CRUD/admin template.

2. Product Personality

The application should feel modern, clean, trustworthy, agricultural but not rustic, professional, calm, efficient, data-driven and friendly.

Avoid: excessive gradients, excessive animation, glassmorphism everywhere, huge decorative illustrations, overly colorful dashboards, dense tables with poor spacing, tiny text, generic Bootstrap-looking interfaces, excessive rounded cards and decorative UI that slows data entry.

3. Technology and Frontend Architecture

- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui where appropriate
- Lucide icons
- React Hook Form
- Zod
- TanStack Query where appropriate
- Responsive layouts
- Accessible semantic HTML
- Component-driven architecture

Integrate with the existing NestJS + PostgreSQL + Prisma + Keycloak backend. Do not replace the existing architecture. Build a real application rather than a static prototype.

4. Design System

Create a complete design system before implementing individual screens.

- Typography, font sizes, weights and line heights
- Spacing scale
- Border radius and shadows
- Icon sizing
- Button hierarchy

- Form controls
- Tables, cards and badges
- Alerts, toasts, dialogs and drawers
- Empty, loading and error states
- Skeleton states

Use a restrained professional color system. Primary should be deep green/teal. Supporting colors: success green, warning amber, error red, information blue and neutral slate/gray. Never use color as the only status signal; combine color with icon and label.

5. Responsive Product Strategy

Desktop

Primary users: Platform Administrators, Cooperative/Cooler Managers and users handling reporting, configuration, finance, audit and operations.

Mobile

Primary users: Milk Collectors, Field Operators and Farmers. The collector experience must be mobile-first and purpose-built rather than simply being a shrunk desktop UI.

Design deliberately for 320px, 375px, 390px, 430px, 768px, 1024px, 1280px and 1440px+.

6. Global Application Shell

Desktop uses a polished left sidebar and top bar containing current cooperative, current cooler, global search, notifications, sync status and user profile.

Navigation

- Overview: Dashboard
- Operations: Collections, Farmers, Collectors, Coolers, Collection Centres, Scale Devices
- Finance: Milk Pricing, SMS Credits, Payments
- Reporting: Reports, Exports
- Management: Users, Roles & Permissions, Applications, Settings
- Governance: Corrections, Reversals, Audit Logs, Security Events

Navigation must dynamically adapt to permissions. Do not show users features they cannot access.

7. Tenant and Cooler Context

Tenant context is critical. Always make the current cooperative and cooler obvious. Context switching should show cooperative name/logo, cooler name, location and status. Users assigned to multiple cooperatives or coolers must be able to switch safely.

8. Global Search

Implement command-palette search using Cmd/Ctrl + K. Search Farmer, Member Number, Cooler, Collection, Collector, Payment, SMS and Application. Group results by entity type.

9. Dashboard

Do not create a wall of statistics. Build an operational overview with greeting, current cooperative, cooler and date.

- Today's Milk
- Monthly Milk
- Active Farmers
- Collections Today

- SMS Credits
- SMS Success Rate
- Pending Corrections
- Pending Payments

Visualizations

- Milk collection trend: Today, 7 days, 30 days, 3 months, Custom
- Cooler performance
- Farmer activity
- SMS health: sent, delivered, failed and pending
- Actionable operational alerts

10. Collection Experience

This is the most important workflow. Make it exceptionally simple and fast.

- 1. Select Cooler
- 2. Connect Scale
- 3. Capture Total Weight
- 4. Assign Milk to Farmers
- 5. Confirm
- 6. Receipt / Sync Status

Scale Interface

Use a large central weight display such as 70.0 KG. Show scale name and Bluetooth connection state. Provide Tare, Capture Weight and Manual Entry. Show a subtle Stable weight detected state.

11. Multi-Farmer Allocation

A physical batch may contain milk from multiple farmers. The system must visually communicate the relationship between total, allocated and remaining quantity.

TOTAL RECEIVED
70 KG

Farmer A 10 KG
Farmer B 40 KG
Farmer C 10 KG
Farmer D 10 KG

Allocated: 70 KG
Remaining: 0 KG

✓ Allocation complete

Green indicates allocated equals total. Amber indicates remaining quantity. Red indicates allocation exceeds total. Confirm must be disabled when invalid. Frontend validation is for UX only; backend validation remains authoritative and atomic.

Farmer Search

Search by member number, name or phone. Results show farmer name, member number and masked phone number. Selecting a farmer should allow immediate quantity entry without unnecessary modal steps.

12. Collection Confirmation and Success

Before submission show cooler, total milk, number of farmers, allocations and SMS receipt count. After success show collection ID, SMS status, sync status and actions for New Collection and View Collection.

13. Offline-First Experience

Offline operation is critical. Never make the application feel broken when connectivity is unavailable.

ONLINE ● Connected
OFFLINE ● Offline
SYNCING ⚡ Syncing
SYNCED ✓ Synced
SYNC ERROR ! Sync needs attention

Create a Sync Center showing synced, waiting and failed records. When connectivity returns, show a subtle confirmation. Do not block normal work merely because the device is offline.

14. Sync Queue

Each queue record should show collection reference, total weight, farmer count and status: Waiting, Syncing, Synced or Failed. Failed records must show the reason and provide Retry. Duplicate offline retries must not create duplicate collections.

15. Farmer Experience

Farmers receive a simpler interface focused on their own records: Today's Milk, This Month, Recent Collections and collection history.

Collection detail should show date, time, cooler, quantity, price, status and receipt. Farmers must only see their own data.

16. Collector Experience

Collectors need speed. Mobile navigation: Home, Collections, Farmers, Sync and More. Make New Collection the dominant action. Use large touch targets, preferably 48px or larger for critical actions.

17. Manager Experience

Manager workflows emphasize milk collection, farmers, coolers, collectors, pricing, SMS credits, corrections, reversals and payments. Quick actions: New Farmer, Add Collector, Add Cooler, Set Monthly Price and Buy SMS Credits.

18. SMS Credits

Build a polished SMS Credit Center. Credit balances are ledger-derived and must not be directly editable.

Available 12,450
Reserved 120
Consumed 8,430

SMS credit health
 68%

[\[Buy Credits\]](#) [\[Purchase History\]](#) [\[Usage\]](#)

Purchase Flow

- Choose package
- Enter M-Pesa confirmation code and amount
- Review package, amount and masked reference
- Submit for payment verification
- Credits are issued only after verification

Never expose the complete M-Pesa confirmation code unnecessarily.

19. SMS Credit Ledger

Provide a financial-style ledger with Date, Type, Reference, Credits, Balance, Status and Actor. Types include Purchase, Issued, Reserved, Consumed, Released, Refunded, Transfer and Adjustment.

20. Payment Verification

Platform administrators need a dedicated verification queue showing applicant, cooler, package, amount, masked M-Pesa reference, submission time and status. Actions: Verify, Request Information and Reject. Verification must never accidentally issue credits.

21. Corrections and Reversals

Corrections show original value, requested value, reason, requester and Approve/Reject actions. Original records must never be overwritten.

Reversals require stronger warnings, maker-checker workflow and confirmation. The requester cannot approve their own reversal.

22. Farmer Management

Create a modern farmer directory with Farmer, Member Number, Phone, Status, Total This Month, Last Collection and Actions. Provide search, filters, sorting, pagination and export. Selecting a farmer opens a detail page or drawer.

23. Cooler Management

Coolers are first-class entities. Cards show cooler name, location, status, today's milk, active collectors, SMS credits and scale status.

Ridgeways Cooler

● Operational

Today 1,240 KG

Collectors 8

SMS Credits 8,420

Scale ● Connected

[Open Cooler]

24. Scale Device UI

- Device name
- Bluetooth status
- Battery
- Last connection
- Firmware where applicable
- Assigned cooler
- Last reading

25. Reporting

Create a professional Reporting Center.

- Daily Collection
- Weekly Collection
- Monthly Collection
- Farmer History
- Cooler Performance
- Collector Performance

- SMS
- SMS Credits
- Payments
- Corrections
- Reversals
- Pricing
- Synchronization

Support date range, cooler, farmer, collector and status filters. Provide CSV and Excel export. Use skeleton loaders while generating.

26. Data Table UX

- Sticky headers
- Sortable columns
- Column visibility
- Search
- Filters
- Pagination
- Export
- Row actions
- Responsive mobile representation

On mobile, convert tables into cards instead of forcing horizontal scrolling wherever practical.

27. Forms and Validation

Every field should have a clear label, input, helper text where needed and validation. Preserve input when validation fails. Prefer inline errors and clear correction guidance. Never expose technical errors such as HTTP 500 to ordinary users.

28. Loading, Empty and Error States

Never leave blank screens. Use skeleton loaders for dashboards, tables, farmer profiles, cooler profiles and reports. Buttons should show Saving, Submitting, Verifying and Synchronizing states.

Every empty state must explain what is empty, why it matters and what the user can do next. Every error must tell the user what to do next.

29. Notifications

Build a notification center with Collections, SMS, Payments, Corrections, Reversals, System and Security categories. Notifications should deep-link to the relevant screen.

30. Confirmation Dialogs

Use confirmation dialogs for meaningful actions such as delete, reverse, approve, reject, issue credits, transfer credits, change pricing and deactivate user. Do not use them for trivial actions.

31. Animation and Motion

Use 150–250ms transitions, subtle hover states, smooth drawers, toast transitions, progress animations, number transitions, success check animations and sync indicators. Avoid long or decorative motion. Respect prefers-reduced-motion.

32. Accessibility

Target WCAG 2.2 AA. Implement keyboard navigation, visible focus states, semantic HTML, ARIA where necessary, sufficient contrast, screen reader labels, accessible dialogs/tables/form errors and reduced motion. Never rely solely on color.

33. Security UX

Security-sensitive actions must communicate risk clearly. For reversal, explain what changes, who requested it and what records are affected. Never display passwords, access tokens, OTPs or secrets. Mask sensitive payment references where appropriate.

34. Mobile Navigation

Collector mobile navigation: Home, Collections, Farmers, Sync and More. Primary action: + New Collection. New Collection must be reachable within one tap.

35. Desktop Navigation

Use a collapsible sidebar. Expanded mode shows icon and label. Collapsed mode shows icon only with tooltip. Remember the user's navigation preference.

36. Microcopy

Use plain language. Prefer “Buy SMS Credits” instead of “Initiate Credit Acquisition”, “Pending verification” instead of “Awaiting administrative validation” and “Collection saved” instead of “Transaction persisted successfully”.

37. Core Interaction Principles

- Every action provides feedback.
- Every destructive action provides context.
- Every loading state explains what is happening.
- Every error tells the user what to do next.
- Every empty state provides a next action.
- Do not make users enter information the system already knows.
- Preserve user input when validation fails.
- Avoid unnecessary confirmation dialogs.
- Keep primary actions visually obvious.
- Keep secondary actions visually quiet.
- Never hide important status information.
- Never expose technical complexity unless the user is an administrator.

38. Performance

- Optimistic UI where safe
- Skeleton loading
- Pagination
- Debounced search
- Virtualized large lists where appropriate
- Lazy loading
- Image optimization
- Code splitting
- Query caching
- Offline caching where appropriate

Do not sacrifice correctness for perceived speed.

39. Reusable Component Library

- AppShell
- Sidebar
- TopBar
- TenantSwitcher
- CoolerSwitcher
- GlobalSearch
- NotificationCenter
- UserMenu
- StatCard
- ChartCard
- DataTable
- MobileDataCard
- StatusBadge
- EmptyState
- ErrorState
- LoadingState
- ConfirmDialog
- Drawer
- FormField
- SearchInput
- FilterBar
- DateRangePicker
- FarmerSelector
- AllocationEditor
- WeightDisplay
- ScaleStatus
- SyncIndicator
- SyncQueue
- CollectionSummary
- CollectionReceipt
- CreditBalance
- CreditLedger
- PaymentVerificationCard
- ApprovalQueue
- AuditTimeline
- ActivityTimeline
- Toast
- CommandPalette

40. Required Screens

Public

- Landing page
- Cooperative/Cooler application
- Application status

- Login
- Registration
- Password reset
- MFA

Platform Admin

- Admin dashboard
- Applications
- Cooperatives
- Coolers
- Users
- SMS packages
- SMS payments
- SMS credit management
- Payment verification
- Audit logs
- Security events
- System settings

Manager

- Dashboard
- Farmers
- Farmer profile
- Collectors
- Coolers
- Collection centres
- Collections
- Collection detail
- Corrections
- Reversals
- Pricing
- SMS credits
- Payments
- Reports
- Settings

Collector

- Mobile dashboard
- New collection
- Scale connection
- Weight capture
- Farmer allocation
- Collection confirmation
- Collection success
- Collection history
- Sync queue
- Farmer search

Farmer

- Farmer dashboard
- Collection history
- Collection detail
- Profile

41. Public Application

The public application should be simple and short.

Headline: “Manage milk collections with confidence.”

Explain that the platform tracks collections, manages farmers, monitors coolers, works offline, sends SMS receipts and manages pricing and reports.

Fields: Name, Organization, Email, Phone, ID Number, KRA PIN, Location and Additional Information. Show application progress.

42. Visual Quality Bar

The product should have the polish associated with modern SaaS products such as Linear, Stripe, Vercel, Notion, Ramp and Shopify Admin, without copying their branding.

- Excellent spacing
- Strong typography
- Clear hierarchy
- Fast interaction
- Excellent empty states
- Excellent tables
- Excellent responsive behavior
- Strong keyboard interaction
- Predictable navigation

43. Domain-Specific UI Requirement

Do not build a generic admin template. The interface must feel specifically designed for milk collection operations.

- Milk weight
- Cooler
- Farmer allocation
- Scale connection
- Collection batch
- SMS credit
- Sync queue
- Milk pricing
- M-Pesa verification

44. Data Integrity UX

The UI must make invalid operations difficult.

TOTAL: 70 KG
 ALLOCATED: 65 KG
 REMAINING: 5 KG

⚠ 5 KG still needs to be allocated.

[Confirm Collection] ← disabled

When allocated equals total, show “✓ All 70 KG allocated” and enable confirmation. Frontend validation improves usability but backend validation remains authoritative.

45. Auditability

Important records should expose an activity timeline.

```
10:32 Collection created — 70 KG
10:33 SMS receipts queued
10:34 Collection synchronized
10:40 Correction requested
11:05 Correction approved by Manager
```

46. Implementation Method

- Define the design system.
- Build the application shell.
- Build the collector workflow.
- Build the manager dashboard.
- Build farmer management.
- Build cooler management.
- Build SMS credits.
- Build payments.
- Build corrections and reversals.
- Build reports.
- Build administration.
- Perform responsive refinement.
- Perform accessibility review.
- Perform interaction review.
- Perform empty/loading/error state review.

47. AI Build Instructions

Do not generate the UI screen-by-screen in isolation. First establish a coherent product design system and interaction model, then implement screens using reusable components.

- Before implementing a new component, check whether an existing component can be reused.
- Before creating a new workflow, check whether it conflicts with existing domain requirements.
- Do not invent business rules.
- Where a requirement is defined, follow it exactly.
- Where visual treatment is unspecified, use sound UX judgment.

Priority: 1. Correct workflow; 2. Usability; 3. Data integrity; 4. Accessibility; 5. Performance; 6. Visual polish; 7. Animation.

48. Final Quality Checklist

- Collector can create a collection quickly
- Collector can work offline
- Offline state is obvious
- Sync state is obvious
- Multi-farmer allocation is intuitive
- Allocation cannot exceed total
- Collection cannot confirm before exact allocation
- Farmer search is fast

- Farmer only sees their own data
- Cooler context is always obvious
- Tenant context is always obvious
- SMS credit balance is clear
- M-Pesa verification is clear
- SMS failures are actionable
- Corrections are easy to review
- Reversals have appropriate warnings
- Reports are easy to generate
- Tables work well
- Mobile UI is purpose-built
- Desktop UI is efficient
- Loading states exist
- Empty states exist
- Error states exist
- Accessibility is addressed
- Keyboard navigation works
- Sensitive information is not unnecessarily exposed
- Backend terminology does not leak into normal workflows
- The product looks polished and cohesive

49. Core Product Principle

Make the application feel SIMPLE even though the underlying system is sophisticated. Users should not need to understand PostgreSQL, RLS, Prisma, Keycloak, Redis, BullMQ, idempotency, transactions, OpenTelemetry or SMS-provider complexity. The system should handle that complexity behind a simple, intuitive interface.

Build the UI as if real milk collectors and cooperative managers will use it every day, often on inexpensive Android devices, sometimes with poor connectivity and sometimes under time pressure.